

# Java One-Dimensional Arrays – Declare, Initialize, and Access Arrays

## 1. What is an Array?

An **array** is a collection of **same-type elements** stored in **contiguous memory locations**.

Instead of creating multiple variables:

```
int mark1 = 85;  
int mark2 = 90;  
int mark3 = 78;
```

Use one array:

```
int[] marks = {85, 90, 78};
```

---

## 2. Why Do We Use Arrays?

Arrays help when:

- Storing multiple values of the same type.
- Accessing elements quickly using an index.
- Iterating through data using loops.

Examples:

- Student marks
  - Product prices
  - Monthly sales
  - Temperatures
  - Employee IDs
-

## 3. Declaring an Array

### Preferred Java Style

```
int[] numbers;
```

### Also Valid

```
int numbers[];
```

### Interview Insight

Both are correct.

However,

```
int[] numbers;
```

is the **Java coding standard** because it clearly shows that the **type is "array of int"**.

---

## 4. Initializing an Array

### Method 1: Size First

```
int[] numbers = new int[5];
```

Creates an array of size 5.

Memory:

```
Index : 0   1   2   3   4
Value : 0   0   0   0   0
```

---

## Method 2: Values Directly

```
int[] numbers = {10,20,30,40};
```

Size is determined automatically.

---

## Method 3: Using new

```
int[] numbers = new int[]{10,20,30};
```

Useful when passing arrays directly to methods.

---

# 5. Accessing Elements

Arrays use **zero-based indexing**.

```
int[] numbers = {10,20,30};  
  
System.out.println(numbers[0]);
```

Output

```
10
```

---

# 6. Modifying Elements

```
int[] numbers = {10,20,30};
```

```
numbers[1] = 100;  
  
System.out.println(numbers[1]);
```

Output

```
100
```

---

## 7. Array Length

Every array has a built-in property:

```
numbers.length
```

Example

```
for(int i=0;i
```

---

## 8. Default Values

When created using new, Java initializes elements automatically.

| Data Type | Default Value |
|-----------|---------------|
| int       | 0             |
| double    | 0.0           |
| boolean   | false         |
| char      | '\u0000'      |
| String    | null          |

---

### Example

```
String[] names = new String[3];  
  
System.out.println(names[0]);
```

### Output

```
null
```

---

## 9. How Arrays Work in Memory

```
int[] arr = new int[3];
```

### Memory:

```
Stack  
-----  
arr  
|  
|  
v  
  
Heap  
-----  
Index 0 -> 0  
Index 1 -> 0  
Index 2 -> 0  
-----
```

### Interview Insight

The **array object** is stored on the **Heap**, while the **reference variable** (**arr**) is stored on the **Stack** (for local variables).

---

This is the same concept you learned with String and other reference types.

---

## 10. Common Mistakes

### ❑ Accessing Invalid Index

```
int[] arr = {10,20,30};  
  
System.out.println(arr[3]);
```

Result

```
ArrayIndexOutOfBoundsException
```

Valid indexes:

```
0  
1  
2
```

---

### ❑ Negative Index

```
arr[-1];
```

Runtime Exception.

---

### ❑ Forgetting Zero-Based Indexing

First element is:

```
arr[0]
```

Not

```
arr[1]
```

---

## 11. Arrays are Fixed in Size

```
int[] arr = new int[5];
```

You **cannot change its size** later.

Need a larger array?

Create a new one.

### Interview Insight

#### Why are arrays fixed in size?

Because Java allocates **contiguous memory** for arrays when they are created. Increasing the size later would require a larger contiguous memory block, which may not be available. Instead of resizing in place, Java creates a new array and copies the elements (this is what classes like ArrayList handle internally).

---

## 12. Array vs Normal Variables

Without Array

```
int a,b,c,d,e;
```

With Array

```
int[] numbers = new int[5];
```

#### Advantages

- Easier iteration
  - Cleaner code
  - Better scalability
- 

### 13. Interview Comparisons

#### Array vs Variables

| Variables                       | Array                      |
|---------------------------------|----------------------------|
| Store one value                 | Store multiple values      |
| Difficult to manage many values | Easy to manage many values |
| No indexing                     | Uses indexing              |

---

#### Array vs ArrayList

| Array                      | ArrayList                                       |
|----------------------------|-------------------------------------------------|
| Fixed size                 | Dynamic size                                    |
| Faster                     | Slightly slower                                 |
| Stores primitives directly | Stores objects (wrapper classes for primitives) |
| Part of Java language      | Part of Collections Framework                   |

#### Interview Insight

Use **arrays** when the size is known and fixed. Use **ArrayList** when the number of elements can change during runtime.

---

## 14. Tricky Interview Questions

Q1. Where is an array stored?

Answer:

The array object is stored on the **Heap**, while its reference variable is stored on the **Stack** (for local variables).

---

Q2. Are arrays objects in Java?

Answer:

Yes. Every array is an object.

Example:

```
int[] arr = new int[5];
```

arr is an object reference.

---

Q3. Can an array store different data types?

Answer:

No. An array stores elements of **one declared type**.

```
int[] arr = {1,2,3};
```

Not

```
int[] arr = {1,"Java"};
```

---

Q4. Is length a method?

Many candidates say **Yes**.

Correct answer:

No.

```
arr.length
```

is a **property (field)**.

Whereas

```
str.length()
```

is a **method**.

---

**Q5. What happens if you access an index outside the array?**

**Answer:**

```
ArrayIndexOutOfBoundsException
```

---

**Q6. Can an array size be changed?**

**Answer:**

No.

Arrays are fixed in size after creation.

---

**Q7. Why do arrays start from index 0?**

**Answer:**

Because array elements are accessed using **offsets** from the starting memory address. The first element is at an offset of 0, making index 0 the natural starting point. This design is efficient and is used by Java and many other programming languages.

---

**Q8. Can you create an array of size 0?**

```
int[] arr = new int[0];
```

**Answer:**

Yes.

It's a valid array with no elements.

---

**Q9. What happens here?**

```
int[] arr = null;  
  
System.out.println(arr.length);
```

**Answer:**

NullPointerException

The reference points to nothing, so there is no array object to access.

---

**Q10. Is this valid?**

```
int[] arr;  
  
arr = new int[5];
```

**Answer:**

Yes.

Declaration and initialization can be done separately.

---

## 15. Output-Based Questions

### Question 1

```
int[] arr = {10,20,30};  
  
System.out.println(arr[1]);
```

Output

20

---

Question 2

```
int[] arr = new int[3];  
  
System.out.println(arr[2]);
```

Output

0

---

Question 3

```
String[] arr = new String[2];  
  
System.out.println(arr[0]);
```

Output

null

---

Question 4

```
int[] arr = {5,10,15};  
  
arr[1] = 100;  
  
System.out.println(arr[1]);
```

Output

100

---

Question 5

```
int[] arr = new int[0];  
  
System.out.println(arr.length);
```

Output

0

---

Interviewer's Reasoning

When an interviewer asks:

"Why would you use an array instead of an ArrayList?"

They're not looking for "because arrays are fixed."

A stronger answer is:

"If the number of elements is known and won't change, an array is preferable because it has lower overhead, stores primitives directly,

---

and provides fast indexed access. If the collection needs to grow or shrink dynamically, ArrayList is the better choice."

This demonstrates that you're choosing the data structure based on the problem, not just recalling definitions.

---

### Cheat Sheet

| Concept                      | Key Point                      |
|------------------------------|--------------------------------|
| Index starts at              | 0                              |
| Last index                   | length - 1                     |
| Size changes after creation? | <b>✗</b> No                    |
| Array stored in              | Heap                           |
| Reference variable stored in | Stack (local variables)        |
| Invalid index                | ArrayIndexOutOfBoundsException |
| length                       | Property (not a method)        |
| Default int value            | 0                              |
| Default String value         | null                           |

---

### Top Interview Questions to Memorize

1. Why are arrays fixed in size?
2. Where are arrays stored in memory?
3. Are arrays objects in Java?
4. Why do arrays use zero-based indexing?
5. What is the difference between length and length()?
6. Can an array store multiple data types?
7. What exception occurs for an invalid index?
8. Can you create an array of size 0?
9. When should you use an array instead of an ArrayList?
10. What are the default values of array elements?

